Project: A Cat vs Dog Classifier with a CNN

Contents

1. Overview . . . 2
2. Prerequisites . . . 2
3. The dataset . . . 3
4. Loading the photos in batches . . . 4
    4.1 Why batches . . . 4
    4.2 `image_dataset_from_directory` . . . 5
5. Scaling the pixels . . . 6
6. The CNN . . . 7
7. Training, and overfitting . . . 9
8. Reducing overfitting . . . 10
    8.1 What the mistakes look like . . . 12
9. Predicting a new photo . . . 13
10. Summary . . . 15
11. Sources . . . 15
12. Key terms . . . 16

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Keras workflow; Scaling inputs for neural networks; CNN architecture (LeNet-5).
- **Leads to:** Data augmentation.
- **Compare with:** Transfer learning (feature extraction and fine-tuning).

1. Overview

Key point: We build a CNN that looks at a photo and says "cat" or "dog", and train it on about 19,000 real photos. It learns quickly, but it **overfits** (**overfitting**, G-1429): after 10 epochs (passes over the training photos) its training accuracy is 99.1% while its validation accuracy (accuracy on photos held back from training) is only 78.9%. **Batch normalisation** (G-266) and dropout narrow this gap only a little (0.20 to 0.17).

This Note puts the CNN Notes into practice. The task is **binary image classification** (G-306): the input is a colour photo, and the **target** (G-1949; the output we predict) is one of two classes, cat or dog. The steps:

1. load the photos from folders, in **batches** (G-263) (sections 3 and 4);
2. scale their pixels (section 5);
3. design a CNN of our own (section 6);
4. train it and watch it overfit (section 7);
5. try two remedies (section 8);
6. use the network on new photos (section 9).

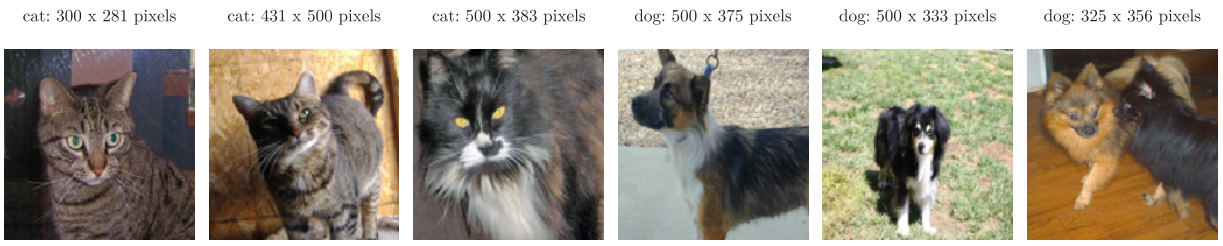


Figure 1: Six photos from the dataset with their label and original size. Every photo has a different size, so all are resized to 256 × 256 before training

2. Prerequisites

- The general CNN architecture: convolution and pooling blocks, Flatten, dense layers.
- **Strides** (G-1900) and **max pooling** (G-1182): the size of each layer's output.
- Building a network in Keras: building, compiling and training a Keras model.

- Why unscaled inputs break training: why inputs are scaled.
- Ways to reduce overfitting, **dropout** (G-639) and batch normalisation: ways to reduce overfitting.

3. The dataset

> **Key point:** 25,000 photos of cats and dogs from Kaggle, in one folder per class; after removing damaged files, 23,410 remain, split 80% for training and 20% for validation.

We use the **Kaggle Cats vs Dogs** dataset (Keras documentation, Image classification from scratch), which Microsoft distributes as a download; its readme points to Microsoft Research's Asirra project. The download holds 12,500 photos of cats and 12,500 of dogs, organised as one folder per class:

```
PetImages/
    Cat/   0.jpg  1.jpg  2.jpg ...
    Dog/   0.jpg  1.jpg  2.jpg ...
```

Each photo is one **observation** (G-1374; one record of the data). Its class is not written in a table: the folder it sits in is its label.

A small number of files are damaged or are not real JPEG photos. The official Keras example deletes every file whose header lacks the "JFIF" marker of a JPEG (Keras documentation, Image classification from scratch); doing the same removes 1,590 files, the same number the example reports, and leaves 23,410 photos: 11,741 cats and 11,669 dogs.

The photos come in many sizes. Among 300 photos picked at random, the widths run from 120 to 500 pixels and the heights from 100 to 500, and there are 200 different sizes (Notebook). A CNN expects every input to have the same shape, so every photo will be resized to one size.

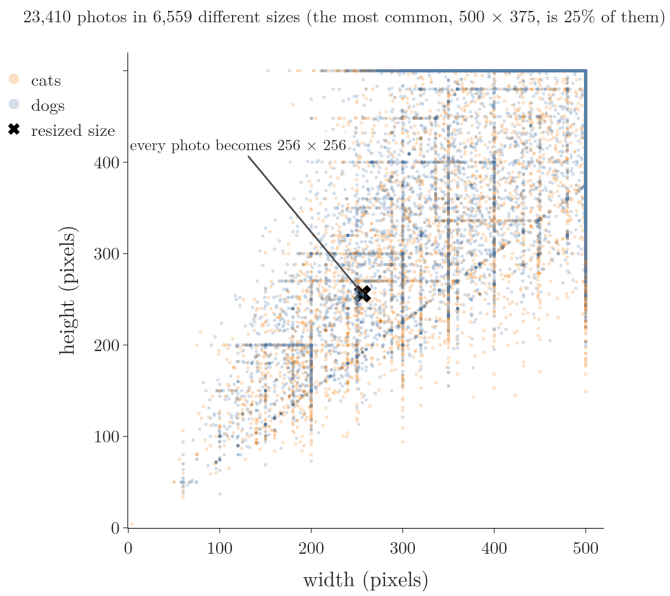


Figure 2: The width and height of all 23,410 photos, one dot each, and the single size every photo is resized to.

Figure 2 is a scatter plot: each dot is one photo, placed by its width (across) and height (up); orange dots are cats, blue dots are dogs, and the black cross marks 256 × 256. Over the whole dataset there are 6,559 different sizes. The most common, 500 × 375, covers a quarter of the photos; no photo is wider or taller than 500 pixels.

Training a CNN on photos takes a lot of computation, and a GPU makes it much faster (see hardware). Without a GPU of our own, a free online notebook service such as Google Colab offers one.

4. Loading the photos in batches

> **Key point:** `image_dataset_from_directory` reads the folders, labels each photo by its folder, resizes it, and serves the photos in **batches** (G-263) of 32, so the whole dataset never has to fit in memory at once.

4.1 Why batches

> **Key point:** All the training photos at 256 × 256 would need 3.7 GB as whole numbers and 14.7 GB as decimal numbers.

We could read every file ourselves, put all photos in one big array and train on it. The problem is memory.

1. **In words:** each photo is 256 × 256 pixels with 3 colour values each. Stored as bytes (whole numbers 0 to 255) each value takes 1 byte; as the 32-bit decimal numbers a network computes with, 4 bytes.

2. **Formula:**

$$\text{values per photo} = 256 \times 256 \times 3 = 196{,}608$$

$$\text{memory} = \text{photos} \times 196{,}608 \times \text{bytes per value}$$

3. **Example:** with 18,728 photos, as bytes:

$$18{,}728 \times 196{,}608 = 3.68 \text{ billion bytes}$$

That is about 3.7 GB. As 32-bit numbers (4 bytes each):

$$4 \times 3.68 = 14.7 \text{ GB}$$

Instead, Keras reads the photos in **batches** (G-263): small groups, here 32 photos. Only the current batch has to be in memory; when the network has learned from it, the next batch is read. An object that hands out data piece by piece in this way is called a **generator** (G-843).

4.2 `image_dataset_from_directory`

Key point: `image_dataset_from_directory` (G-94): give it the folder, how to label, the batch size and the image size; it returns a dataset of (images, labels) **batches** (G-263).

Python: Training and validation datasets from one folder.

```
import keras
train_ds, val_ds = keras.utils.image_dataset_from_directory(
    "PetImages", labels="inferred", label_mode="int",
    batch_size=32, image_size=(256, 256),
    validation_split=0.2, subset="both", seed=1337)
```

`labels="inferred"` takes each photo's label from its folder. With `label_mode="int"` the folders get the numbers 0, 1, ... in alphabetical order: `Cat` is 0 and `Dog` is 1. `image_size=(256, 256)` resizes every photo, so each batch has the shape $(32, 256, 256, 3)$. `validation_split=0.2` with `subset="both"` keeps 20% of the photos apart as the **validation set** (G-2067); the `seed` fixes which ones.

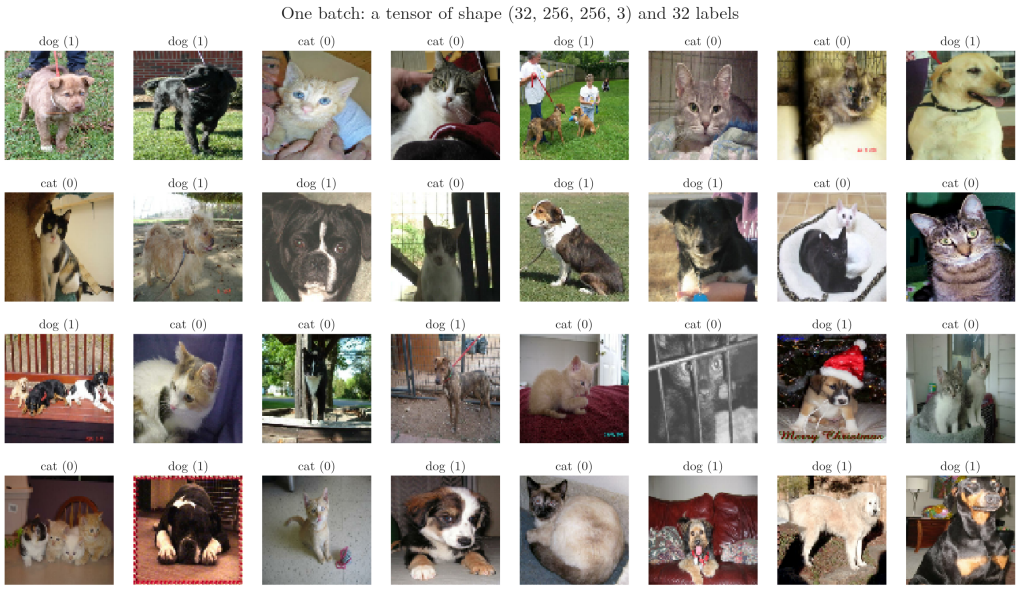


Figure 3: The first training batch served by the call above: 32 photos, resized to 256 × 256 (drawn smaller here), each labelled by its folder.

Figure 3 shows the first batch it serves. Its title calls the batch a **tensor**: an array with several axes, here 32 photos × 256 rows × 256 columns × 3 colour values. The batch holds 32 photos of all kinds, now all square, with labels 0 (cat) and 1 (dog), 16 of each here. The square resize stretches photos that were not square, which the network has to live with.

The split gives 18,728 training photos and 4,682 validation photos (Notebook). With 32 photos per batch, one pass over the training photos, an **epoch** (G-696), takes 586 batches. The last batch is only partly full, so the division is rounded up ($\lceil x \rceil$ means "round x up"):

$$18{,}728/32 = 585.25$$
$$\lceil 585.25 \rceil = 586$$

The function's defaults are `batch_size=32` and `image_size=(256, 256)` (Keras documentation, `image_dataset_from_directory`).

5. Scaling the pixels

> **Key point:** Divide every pixel by 255 so that inputs lie between 0 and 1.

Pixel values run from 0 to 255. Large, unscaled inputs make gradient descent slow and unstable, as why unscaled inputs break training shows, so we bring every value into the range 0 to 1.

> **Python:** Apply a scaling function to every **batch** (G-263).
> ```
> def process(image, label):
> ```

```
    image = image / 255.0
    return image, label

train_ds = train_ds.map(process)
val_ds = val_ds.map(process)
```

`map` applies `process` to every (images, labels) pair the dataset produces. The labels pass through unchanged.

6. The CNN

Key point: Three blocks of convolution (32, 64, 128 **filters**, G-777) and **max pooling** (G-1182), then Flatten and dense layers of 128, 64 and 1 node. Of its 14.8 million parameters, 99.3% sit in the first dense layer.

The design follows the general CNN architecture: convolution and pooling blocks that grow the number of filters while the maps shrink, then fully connected layers (Figure 4).

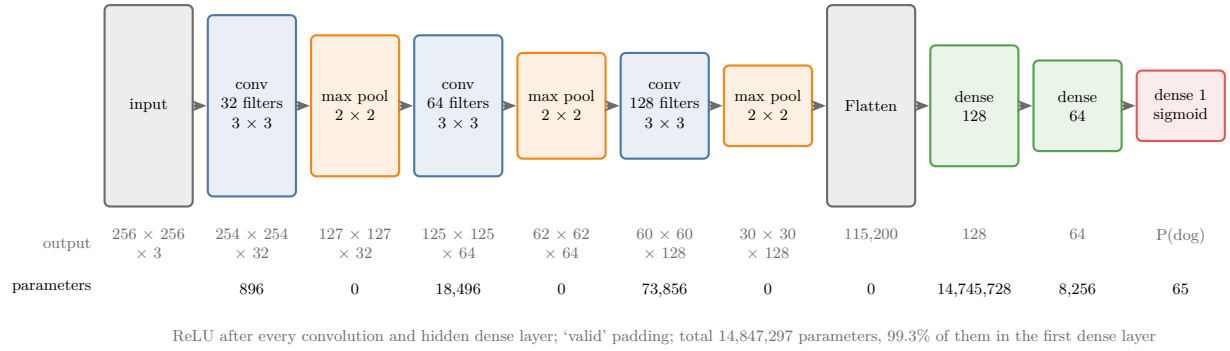


Figure 4: The CNN: three convolution (blue) and max-pooling (orange) blocks, Flatten, and three dense layers. Under each layer: its output size and its number of parameters

Python: The model.

```
from keras import layers
model = keras.Sequential([
    keras.Input(shape=(256, 256, 3)),
    layers.Conv2D(32, kernel_size=(3, 3), padding="valid",
↪   activation="relu"),
    layers.MaxPooling2D(pool_size=(2, 2), strides=2, padding="valid"),
    layers.Conv2D(64, kernel_size=(3, 3), padding="valid",
↪   activation="relu"),
    layers.MaxPooling2D(pool_size=(2, 2), strides=2, padding="valid"),
    layers.Conv2D(128, kernel_size=(3, 3), padding="valid",
↪   activation="relu"),
    layers.MaxPooling2D(pool_size=(2, 2), strides=2, padding="valid"),
    layers.Flatten(),
```

```
    layers.Dense(128, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(1, activation="sigmoid")])
model.summary()
```

The output layer has one node with a **sigmoid** (G-1798): its value is the probability that the photo shows a dog (label 1).

Each output size follows from the formula of **strides** (G-1900): a 3×3 convolution without padding removes 2 pixels, and 2×2 pooling with stride 2 halves the size, rounding down:

$$256 \to 254 \text{ (convolution)}$$

$$254 \to 127 \text{ (pooling)}$$

The second and third blocks repeat the same two steps (Figure 4, row "output"):

$$127 \to 125 \to 62$$

$$62 \to 60 \to 30$$

After three blocks the photo has become $30 \times 30 \times 128$, since the last convolution has 128 filters. Flatten turns it into one row of numbers:

$$30 \times 30 \times 128 = 115{,}200$$

The parameters are counted the same way as in a dense layer: one weight per input value of the filter plus one bias per filter.

1. **In words:** a convolution layer has (filter height × filter width × input channels + 1) × filters parameters; a dense layer has (inputs + 1) × nodes.
2. **Formula** (k = filter size, c = input channels, f = filters, n = inputs, m = nodes):

$$\text{conv: } (k \times k \times c + 1) \times f$$

$$\text{dense: } (n + 1) \times m$$

3. **Example:** the first convolution (3×3 filters on 3 colour channels, 32 filters):

$$3 \times 3 \times 3 = 27 \text{ weights per filter}$$

$$27 + 1 = 28 \text{ (with the bias)}$$

$$28 \times 32 = 896$$

The first dense layer (115,200 inputs, 128 nodes):

$$115{,}200 + 1 = 115{,}201$$

$$115{,}201 \times 128 = 14{,}745{,}728$$

The model has 14,847,297 parameters in total, and the first dense layer alone holds 99.3% of them; the three convolution layers together have only 93,248 (Notebook). The convolution layers share each small filter across the whole photo, while the dense layer connects every one of the 115,200 inputs to every node.

7. Training, and overfitting

Key point: After 10 **epochs** (G-696) the training accuracy is 99.1% but the validation accuracy 78.9%; the validation loss is lowest at epoch 3 and then rises. The model is memorising the training photos.

Python: Compile and train for 10 epochs.

```
model.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=["accuracy"])
history = model.fit(train_ds, epochs=10, validation_data=val_ds)
```

`binary_crossentropy` (**binary cross-entropy**, G-303) is the loss for a two-class problem with a **sigmoid** (G-1798) output (see binary cross-entropy). `history.history` keeps the training and validation accuracy and loss of every epoch, for plotting.

We trained the model three times with different random seeds (each seed gives different starting weights); Figure 5 shows all three runs and their mean.

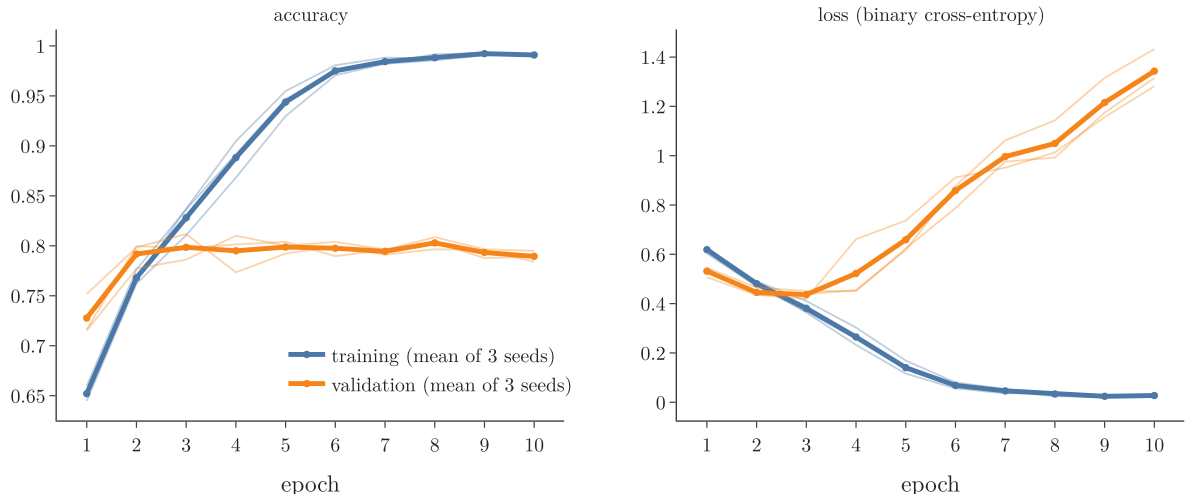


Figure 5: The plain CNN: training (blue) and validation (orange) accuracy and loss over 10 epochs. Thin lines: 3 runs with different seeds; thick lines: their mean

Epoch	Training accuracy	Validation accuracy	Training loss	Validation loss
1	65.2%	72.8%	0.62	0.53
3	82.8%	79.9%	0.38	0.44

Epoch	Training accuracy	Validation accuracy	Training loss	Validation loss
5	94.4%	79.9%	0.14	0.66
7	98.4%	79.4%	0.05	1.00
10	99.1%	78.9%	0.03	1.34

(Means of 3 seeds; Notebook.)

At epoch 1 the validation accuracy (72.8%) is above the training accuracy (65.2%). The training figure is the average over all 586 **batches** (G-263) of the epoch, including the early ones seen while the weights were still poor; the validation figure uses the model as it is at the end of the epoch (Keras FAQ). By epoch 3, training is ahead.

The two curves tell different stories (Figure 5). The training accuracy rises every epoch, from 65% to 99%, and the training loss falls almost to 0. The validation accuracy reaches about 80% by epoch 2 or 3 and then stays there, moving between 79% and 80%. The validation loss is lowest at epoch 3 (0.44) and then climbs every epoch, to 1.34 at epoch 10: on the photos it gets wrong, the model becomes more and more confident (section 8.1 counts these photos). By epoch 10 the gap between training and validation accuracy is 0.20.

Such a growing gap between training and validation performance is **overfitting**: the model fits details of its own training photos that do not hold for new photos (see why neural networks overfit). Early stopping shows how to stop training near the epoch where the validation loss is lowest.

8. Reducing overfitting

> **Key point:** Options: more data, data augmentation, L1/L2 regularisation, dropout, batch normalisation, or a smaller model. Batch normalisation after each convolution and dropout of 0.1 after the dense layers narrows the gap a little and nearly halves the final validation loss (1.34 to 0.76), but does not remove the overfitting.

Ways to reduce overfitting lists the ways to fight it:

- **More data.** The best remedy, but here we already use every photo we have.
- **Data augmentation:** create new training photos from the existing ones, by flipping, rotating or zooming them. **Data augmentation** (G-531) covers it.
- **L1 or L2 regularisation:** penalise large weights.
- **Dropout** (G-639): switch off random nodes during training (see how dropout works).
- **Batch normalisation** (G-266): normalise each layer's values over the **batch** (G-263); it also has a mild regularising effect (see advantages of batch normalisation).
- **A simpler model:** fewer layers or fewer nodes.

We try two of them together. A `BatchNormalization` layer goes after each of the three convolution layers, and a `Dropout(0.1)` layer after each of the two hidden dense layers. Everything else stays the same, and again we train 3 times.

Python: One convolution block and the dense part, with the two additions.

```
layers.Conv2D(32, kernel_size=(3, 3), padding="valid",
↪  activation="relu"),
layers.BatchNormalization(),
layers.MaxPooling2D(pool_size=(2, 2), strides=2, padding="valid"),
# ... the same for the 64- and 128-filter blocks ...
layers.Flatten(),
layers.Dense(128, activation="relu"),
layers.Dropout(0.1),
layers.Dense(64, activation="relu"),
layers.Dropout(0.1),
layers.Dense(1, activation="sigmoid"),
```

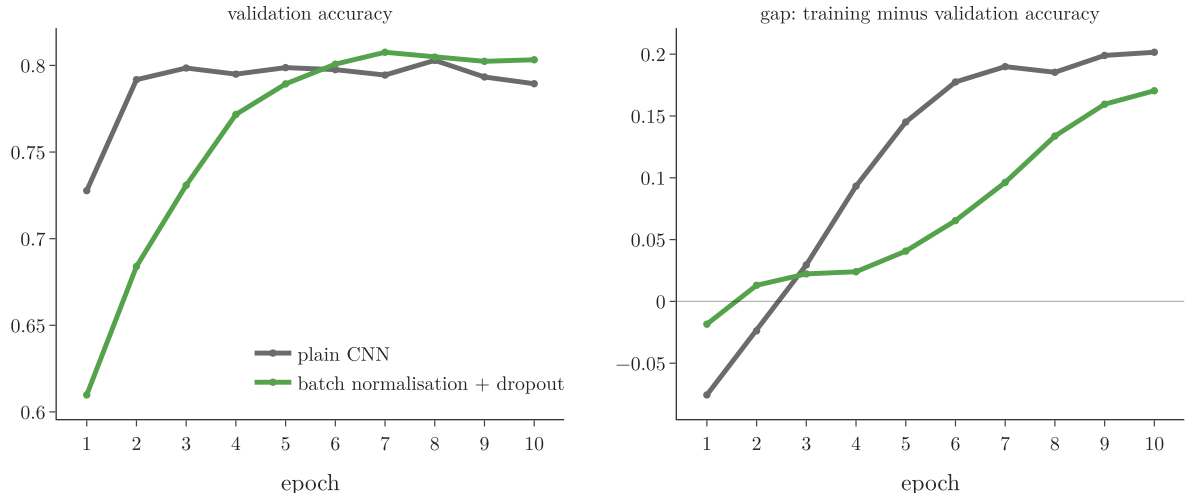


Figure 6: Plain CNN (grey) against the same CNN with batch normalisation and dropout (green): validation accuracy (left) and the gap between training and validation accuracy (right), means of 3 seeds

Mean of 3 seeds	Plain CNN	With batch normalisation + dropout
Validation accuracy, epoch 10	78.9%	80.3%
Best validation accuracy (epoch)	80.3% (8)	80.8% (7)
Training accuracy, epoch 10	99.1%	97.4%
Gap (training minus validation), epoch 10	0.202	0.171
Validation loss: lowest (epoch), then at epoch 10	0.44 (3), then 1.34	0.46 (5), then 0.76

The two additions help, but only a little (Figure 6; Notebook). The validation accuracy at **epoch** (G-696) 10 rises from 78.9% to 80.3%, the gap shrinks from 0.20 to 0.17, and the final validation loss is about half as large (0.76 against 1.34). The model with batch normalisation and dropout learns more slowly at first (61% validation accuracy after one epoch, against 73%), so it starts to overfit later: its gap stays below 0.05 up to and including epoch 5 (Figure 6, right). By epoch 10 it is overfitting too. Its training accuracy is 97.4%, and its validation loss has risen since epoch 5.

Dropout of 0.1 removes only one node in ten, and the mild regularising effect of batch normalisation is a side benefit rather than a cure (see batch normalisation). Stronger remedies for this model are more varied training data, which **data augmentation** (G-531) creates, and stopping near the epoch of the lowest validation loss.

8.1 What the mistakes look like

> **Key point:** The overfitted CNN is not unsure about the photos it gets wrong: in 69% of its mistakes it gives the wrong answer a probability above 0.9. With **batch normalisation** (G-266) and dropout this share falls to 50%.

A rising validation loss with a flat validation accuracy (section 7) means that the wrong answers are becoming more confident. To see it photo by photo, we trained each model once more (seed 0, 10 **epochs** (G-696); script `experiments/mistakes.py`) and kept the probability each gives to every validation photo.

One run of each model	Plain CNN	With batch normalisation + dropout
Validation accuracy	75.7%	78.0%
Validation photos labelled wrongly (of 4,682)	1,140	1,028
… with a probability above 0.9 for the wrong class	789 (69%)	512 (50%)
… with a probability above 0.99 for the wrong class	503	183

(Single runs, so the accuracies differ a little from the 3-seed means above.)



Figure 7: The 12 validation photos the plain CNN gets wrong with the highest confidence. Above each: the true label, the plain CNN's answer and the answer of the CNN with batch normalisation and dropout, each with the probability the model gives to its answer

Figure 7 shows the plain CNN's 12 most confident mistakes. For every one of them it gives the wrong class a probability that rounds to 100%. The regularised model gets 4 of the 12 right, and on some of the others it is less sure (58%, 52%). 467 validation photos are labelled wrongly by both models.

9. Predicting a new photo

> **Key point:** Prepare a new photo exactly like the training photos: resize to 256×256, divide by 255, and add a **batch** (G-263) dimension. A probability above 0.5 means dog.

The network has only ever seen photos as batches of shape $(\text{batch}, 256, 256, 3)$ with values between 0 and 1. A new photo must arrive in the same form.

Python: Predict one photo.

```
import numpy as np
# read and resize
img = keras.utils.load_img("dog.jpg", target_size=(256, 256))
# (256, 256, 3), scaled like training
x = keras.utils.img_to_array(img) / 255.0
# a batch of one: (1, 256, 256, 3)
x = np.expand_dims(x, axis=0)
# probability of "dog"
p = model.predict(x)[0, 0]
print("dog" if p > 0.5 else "cat", p)
```

> `np.expand_dims(x, axis=0)` adds a new first axis of length 1: the model expects a batch, and this batch holds one photo.

We tried two photos from Wikimedia Commons that are not in the dataset: a tabby kitten of 1728×1152 pixels and a golden retriever of 652×515 (Notebook):

Photo	Plain CNN: P(dog)	With batch normalisation + dropout: P(dog)
tabby kitten	$0.000 \rightarrow$ cat	$0.000 \rightarrow$ cat
golden retriever	$0.999 \rightarrow$ dog	$1.000 \rightarrow$ dog

Both models label both photos correctly and with near certainty, even though the validation accuracy is only about 80%: these two photos are clear, close-up views of one animal each. On the 4,682 validation photos, about one in five is still labelled wrongly.

10. Summary

Step	What we did
Data	23,410 photos of cats and dogs, one folder per class; 18,728 for training, 4,682 for validation
Loading	`image_dataset_from_directory`: labels from folders, resized to 256 × 256, batches of 32
Scaling	pixels divided by 255
Model	3 blocks of Conv2D (32, 64, 128) and MaxPooling, Flatten, Dense 128, 64, 1; 14.8 million parameters
Training	Adam, binary cross-entropy, 10 epochs
Result	99.1% training accuracy but 78.9% validation accuracy: overfitting
Remedy	batch normalisation and dropout: 80.3% validation accuracy, gap 0.17 instead of 0.20
New photo	resize, divide by 255, add a batch dimension, predict; above 0.5 means dog

- Images of different sizes must be resized to one shape before they enter a CNN, because a CNN expects every input to have the same shape.
- Loading in **batches** (G-263) keeps memory use small: the full training set would need 14.7 GB as 32-bit numbers.
- Pixels are divided by 255, because large, unscaled inputs make gradient descent slow and unstable.
- In this CNN almost all parameters sit in the first dense layer, not in the convolutions, because a convolution shares each small filter across the whole photo, while the dense layer connects all 115,200 inputs to every node.
- A widening gap between training and validation accuracy, with a rising validation loss, signals overfitting, because the model is fitting details of its own training photos that do not hold for new ones; **batch normalisation** (G-266) and dropout narrowed the gap only a little, so stronger remedies such as data augmentation or early stopping are needed.

11. Sources

Built from

- CampusX, “Cat Vs Dog Image Classification Project | Deep Learning Project | CNN Project”, YouTube, https://www.youtube.com/watch?v=0K4J_PTgysc

Other references

- Keras documentation: `image_dataset_from_directory`, keras.io/api/data_loading/image (arguments and defaults: `labels="inferred"`, `label_mode`, `batch_size=32`,

`image_size=(256, 256)`, `validation_split`, `subset`).

- Keras documentation: Image classification from scratch, keras.io/examples/vision/image_classification (the Kaggle Cats vs Dogs dataset; deleting files without "JFIF" in their header: 1,590 files).
- Microsoft Download Center: Kaggle Cats and Dogs Dataset (kagglecatsanddogs_5340.zip; 12,500 cat and 12,500 dog photos; readme pointing to the Asirra project of Microsoft Research).
- Keras FAQ, "Why is my training loss much higher than my testing loss?" (the training metric is averaged over the epoch's batches; the validation metric uses the model at the end of the epoch).
- Photos for prediction: Wikimedia Commons. Kitten (David Corby, CC BY 2.5); golden retriever (Janneke Vreugdenhil, public domain).

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Binary image classification (G-306)	Sorting photos into one of two classes, such as cat or dog.
`image_dataset_from_directory` (G-94)	The Keras function that loads photos from class folders as batches of (images, labels).
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
Batch normalisation (G-266)	A layer placed between layers of a network that re-centres and re-scales its inputs during training; this keeps the activations away from the flat ends of sigmoid and tanh, which helps against vanishing gradients.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Batch (mini-batch) (G-263)	A small group of training observations used for one update; Keras uses 32 by default.
Stride (G-1900)	The number of pixels the filter moves at each step; a stride larger than 1 makes the output smaller.
Max pooling (G-1182)	Pooling that keeps only the largest value in each window (usually 2×2), which shrinks the feature map while keeping the strongest response in each region.
Dropout (G-639)	Switching off a random set of input and hidden nodes at every training step, to reduce overfitting.

Term	Meaning
Observation (G-1374)	One record of a dataset: one row of the data table.
Generator (G-843)	The GAN network that creates new data.
Validation set (G-2067)	Data held back from training to check and tune a model before the final test.
Epoch (G-696)	One full update of the parameters using the whole training set.
Filter (kernel) (G-777)	A small grid of weights slid over the image; at each position it multiplies and adds, so it responds strongly where its pattern (such as an edge) appears and produces a feature map. Its depth equals the input's number of channels.
Sigmoid function (G-1798)	An S-shaped function that squashes any number into the range 0 to 1, $\sigma(z) = 1/(1 + e^{-z})$; it turns a score into a probability, as in logistic regression.
Binary cross entropy (log loss) (G-303)	The loss function of logistic regression for two classes: per row $-y \log a - (1 - y) \log(1 - a)$, averaged over the rows (the average cross entropy).
Data augmentation (G-531)	Enlarging a dataset by making changed copies of its examples, such as zoomed or shifted images.